

Chapter-2

Elementary Graph operation

graph traversalal method:-

traversal means to visit all the nodes

In the graph graph supports two types of traversal methods they are

* DFS (Depth first search)

* BFS (Breadth first search)

DFS (depth first search):- (LIFO)

DFS follows stack operation It means

last inserted element first deleted i.e; most recently entered element treated as top most element and all nodes arranged in depth wise manner

Algorithm DFS (or)

Step 1: Start

Step 2: begin with topmost node and push into the stack.

Step 3: repeat the steps 4 and 5 until the stack is empty.

Step 4: pop the top most node after stack and print it

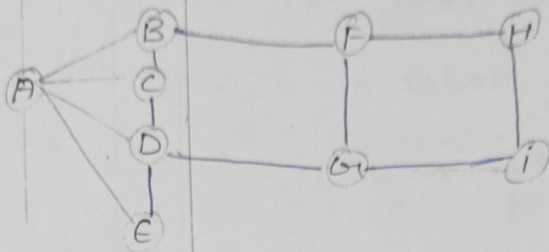
Step 5:

push the adjacent nodes of the popped element P_n to the stack.

Step 6:

isTop()

eg:-



Step 1:

push A into the stack

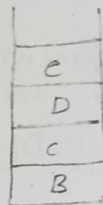
DFS: empty



Step 2:

pop A and print A, and also push the adjacent nodes of 'A' into the stack

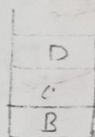
DFS: A



Step 3:

Pop 'e' and print e and also push the adjacent nodes of 'e' into the stack.

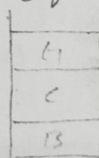
DFS: Ae



Step 4:

pop D and print and also push the adjacent nodes of 'D' into the stack

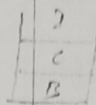
DFS: AeD



Step 5:

pop G and print and also push the adjacent nodes of 'G' into the stack

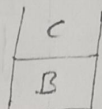
DFS: A C D G



Step 6:

pop D and print and also push the adjacent nodes of D into the stack.

DFS: A C D G D



Step 7:

pop C and print and also push the adjacent nodes of 'C' into the stack.

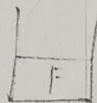
DFS: A C D G D C



Step 8:

pop B and print and also push the adjacent node to 'C' to the stack.

DFS: A C D G D C B



Step 9:

pop F and print and also push the adjacent node to 'F' to the stack

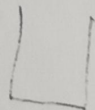
DFS: A C D G D C B F



Step 10:

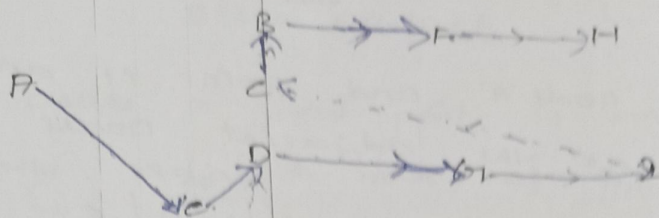
pop H and print and also push the adjacent node to H to the stack

DFS: A C D G D C B F H



The result of DFS traversing is

A → C → D → G → I → E → B → F → H

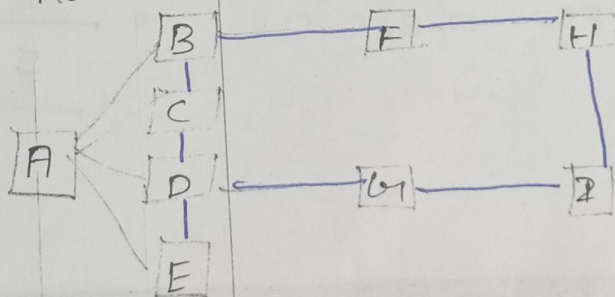


B.F.S (Breadth First Search)

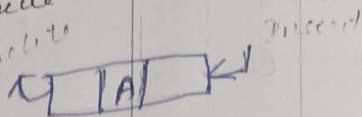
BFS traverse can follow queue operation. It means first in first out (FIFO) in this method. elements are inserted at one end and deleted another end. In this traversal elements are traversed in Breadth wise manner.

Algorithm:-

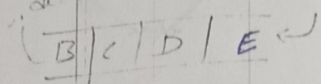
- Step 1: Start
- Step 2: begin at root node and insert that node into the queue
- Step 3: repeat the steps 2 and 3 until the queue is empty.
- Step 4: delete root node and print it
- Step 4: Insert adjacent nodes of a deleted node into the queue.



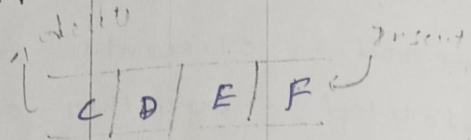
Step 1: Insert 'A' into the queue
BFS: empty



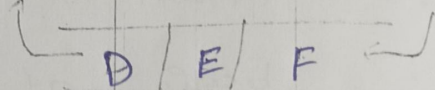
Step 2: Delete node 'A' and point it and also
Insert all the adjacent nodes of 'A'
BFS: A



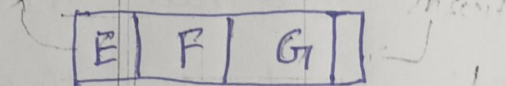
Step 3: Delete node 'B' and point it and also
Insert all the adjacent nodes of
B.
BFS: A, B



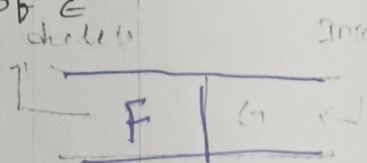
Step 4: Delete node 'C' and point it and also
also Insert all the adjacent nodes
of 'C'
BFS: A, B, C



Step 5: Delete 'D' and point it and also
Insert all the adjacent nodes of 'D'
BFS: A, B, C, D



Step 6: Delete 'E' and point it and also Insert
all the adjacent nodes of E
BFS = A, B, C, D, E



Step 7: Delete 'F' and point it and also insert all the adjacent nodes of F
 BFS = ABCDEF
 $\xrightarrow{\text{delete}}$

G	H
---	---

 $\xleftarrow{\text{insert}}$

Step 8: Delete 'G' and point it and also insert all the adjacent nodes of 'G'
 BFS = ABCDEFG
 $\xrightarrow{\text{delete}}$

H	I
---	---

 $\xleftarrow{\text{insert}}$

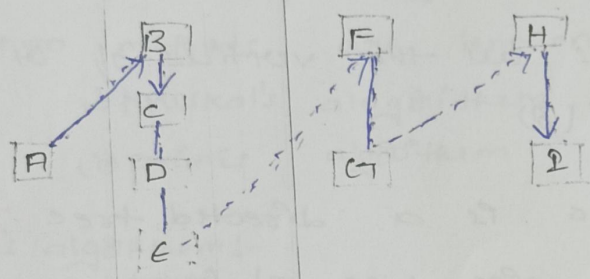
Step 9: delete H and point it and also insert all the adjacent nodes of H
 BFS = ABCDEFGH

I

Step 10: delete I and point it and also insert all the adjacent nodes of I.
 BFS: ABCDEFGHI
 $\xrightarrow{\text{delete}}$

--

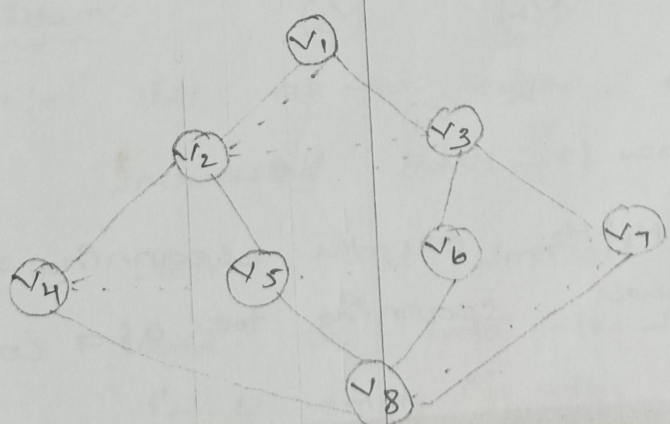
 $\xleftarrow{\text{insert}}$



The result of BFS is

A → B → C → D → E → F → G → H → I

Eg:-



BFS = empty

BFS = v_1

BFS $v_1 v_2$

BFS : $v_1 v_2 v_3$

BFS $v_1 v_2 v_3 v_4$

BFS $v_1 v_2 v_3 v_4 v_5$

BFS $v_1 v_2 v_3 v_4 v_5 v_6$

BFS $v_1 v_2 v_3 v_4 v_5 v_6 v_7$

BFS $v_1 v_2 v_3 v_4 v_5 v_6 v_7 v_8$

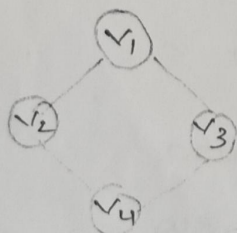
Spanning tree:-

The sub graph H of a graph G is called a spanning tree if

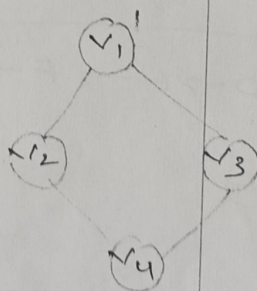
a) H is a tree and

b) H contains all the vertices of G
(8)

A spanning tree is a directed tree is called a directed spanning tree of G .



H_1



H_2

These two H_1 & H_2 are the spanning trees of graph H to produce spanning tree of a connected

graph we use methods called spanning of graph.

(81)

The spanning tree of a graph G can be defined as a tree which includes all the vertices of G in during the discussion

Minimum spanning tree:-

minimum spanning tree of an undirected graph G is a tree formed from graph edges that connects all the vertices of graph at lowest total cost. A minimum spanning tree exist if and only if G is connected.

There are several methods available for finding minimum spanning tree of a graph out of them two methods are known to be very efficient

- (i) Kruskal's algorithm and
- (ii) Prim's algorithm

Kruskal's algorithm:-

let us assume an undirected weighted graph with n vertices where initially the spanning tree is empty

Algorithm:-

Step-1:- list all the edges of the graph G in the increasing order of weights

Step-2:- select the smallest edge from the list and fit into the spanning tree initially if it is empty if the inclusion of this edge

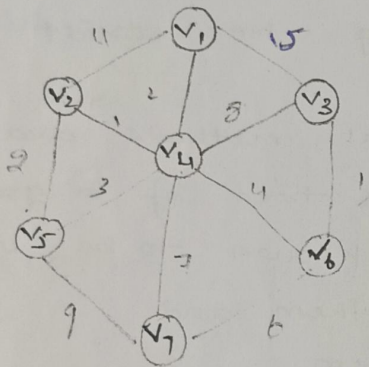
does not make a cycle

Step 3:- If the selected edge with smallest weight from cycle remove it from the list.

Step 4:- Repeat step 3-5 until the tree contains $n-1$ edges (or) list is empty.

Step 5:- If the tree contains less than $n-1$ edges & the list is empty no spanning tree is possible for the graph else return the minimum spanning tree.

The following diagram shows the entire information about the Kruskal's algorithm.



<u>Edge</u>	<u>Weight</u>	<u>Selection</u>
$V_2 \rightarrow V_4$	1	✓
$V_3 \rightarrow V_6$	1	✓
$V_2 \rightarrow V_5$	2	✓
$V_1 \rightarrow V_4$	2	✓
$V_4 \rightarrow V_5$	3	✓
$V_1 \rightarrow V_2$	4	×
$V_4 \rightarrow V_6$	4	×
$V_1 \rightarrow V_3$	5	✓
$V_6 \rightarrow V_7$	6	×
$V_4 \rightarrow V_7$	7	✓
		×

$v_4 \rightarrow v_3$

$v_5 \rightarrow v_7$

8

9

x

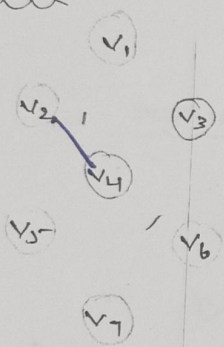
x

let us

consider an example for the

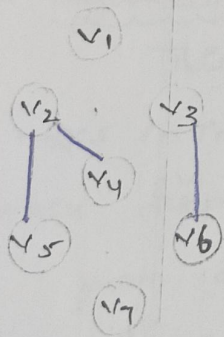
construction of the above algorithm for a graph
 6. A list of edge either their weight are marked
 from this list we have select (✓) or reject (x)
 depending on either a from a cycle (or) the length
 of a spanning tree has option can be calculated
 has 16.

Step 1:-



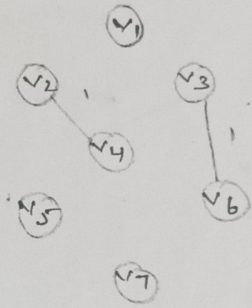
no cycle is form so
 consider the edge v_2 and v_4

Step 3



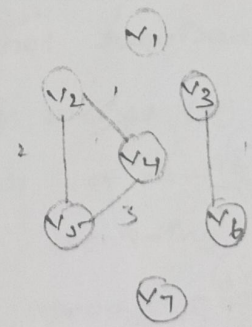
no cycle is form so
 consider edge v_2 and v_5

Step 2:-



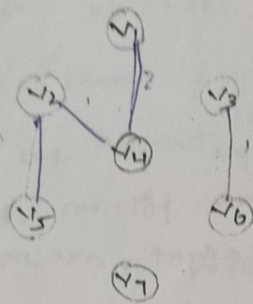
no cycle is form so
 consider the edge v_3 and v_6

Step 4



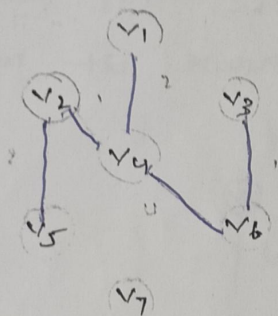
This cycle is form with
 edges v_4 and v_5 and v_2 and v_4
 v_2, v_4 so do not consider the
 edge.

Step 5:-



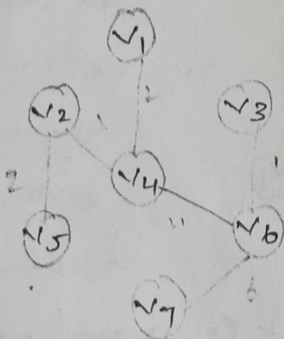
No cycle is form with the edge V_4, V_1 is consider.

Step 7:-



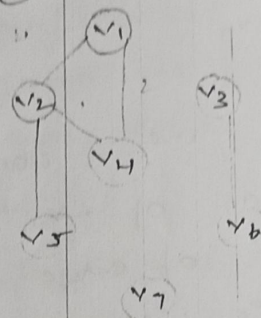
No cycle is form with the edge V_2, V_6 so consider that the edge

Step 9:-



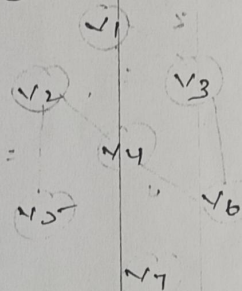
No cycle is form with the edge V_6, V_7 so consider that edge

Step 6:-



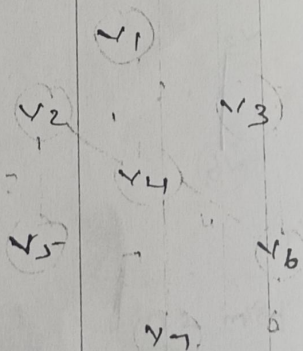
The cycle is form with the edge V_1, V_2 so we need not consider the edge

Step 8:-



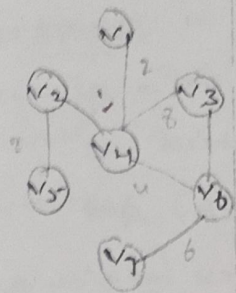
cycle is form with the edge V_1, V_3 so we need not consider the edge V_1, V_3

Step 10:-



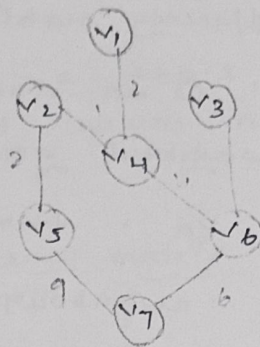
The cycle is form with the edge V_4, V_7 so we need consider the edge V_4, V_7

Step 1:-



The cycle is form with the edge v_3 & v_4 so we need not consider the edge

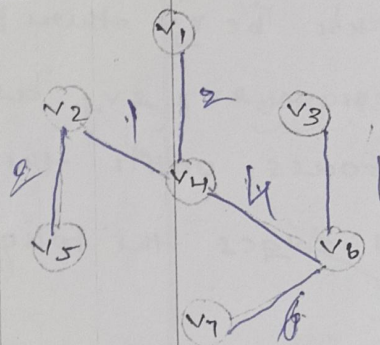
Step 2:-



cycle is form with the edge v_5 & v_7 so we need not consider the edge

Result:- $(v_1 \rightarrow v_4) + (v_2 \rightarrow v_4) + (v_2 \rightarrow v_5) + (v_4 \rightarrow v_6) + (v_7 \rightarrow v_6) + (v_3 \rightarrow v_6)$

result:- $2 + 1 + 2 + 4 + 6 + 1 = 16$ ✓



Prims algorithm:-

This approach does not required listing of all edges in increasing order of weight (or) checking of each step that is whether a newly selected edge from a circuit (or) not according to the prims algorithm. minimum spanning tree grows in successive stages at any stage in the algorithm, we can see that we have a set of vertices that have already been included in the tree the rest of the vertices have not.

The prime algorithm can easily be implemented using the adjacency matrix representation of a graph. Suppose there is an $n \times n$ adjacency matrix for a given undirected weighted graph & vertices are labelled as $v_1, v_2, v_3, \dots, v_n$. Start from vertex ' v_i ' say get its nearest neighbour that is the vertex whose is the smallest entry in row of vertex ' v_i ' (if the minimum one smallest entry are there arbitrarily select any one) let it be v_j now consider v_i, v_j as one sub graph next obtain the closest neighbour of this sub graph i.e., vertex other than v_i, v_j that has the smallest entry among all entries in the rows of v_i, v_j let this new vertex be v_k then join the three now proceed with vertex v_i, v_j, v_k as one sub graph continuous this process until the all vertices have been connected $n-1$ edges the following are the algorithm.

Step 1:-

let G be a connected graph with non -ve values assigned to each edge.

Step 2:-

let ' T ' be the tree consisting of any vertex v_1 (or)

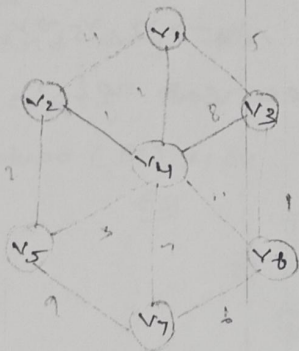
Step 3:-

Among all the edges not in ' T ' that are incident on a vertex in ' T ' and do not form a circuit and added to ' T ' select one of minimum cost and add it.

TO 'T'

Step 4:-

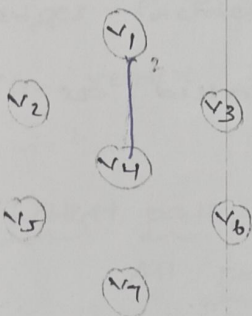
This process terminates i.e. step-3 repeated until $n-1$ edges are selected where n is no. of vertices in 'G'



adjacency matrix

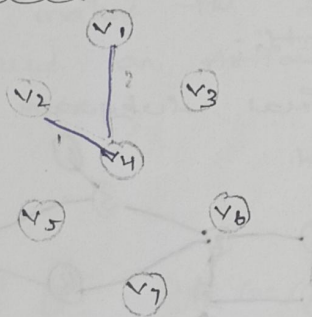
	v_1	v_2	v_3	v_4	v_5	v_6	v_7
v_1	-	4	5	2	-	-	-
v_2	4	-	-	1	2	-	-
v_3	5	-	-	8	-	1	-
v_4	2	1	8	-	3	4	7
v_5	-	2	-	3	-	-	9
v_6	-	-	1	4	-	-	6
v_7	-	-	-	7	9	6	-

Step 1:-



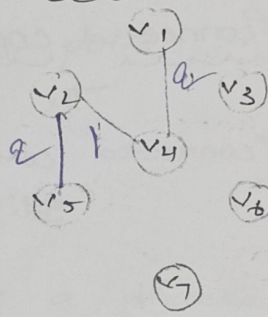
Set (v_1, v_4)

Step 2:-



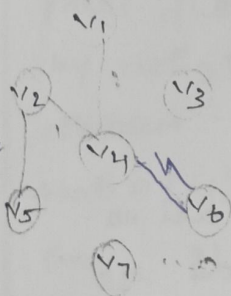
Set (v_1, v_4, v_2)

Step 3:-



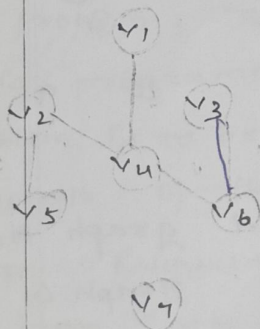
Set (v_1, v_4, v_2, v_5)

Step 4:-



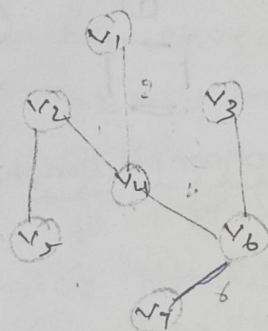
Set $(v_1, v_4, v_2, v_5, v_6)$

Step 5:-



Set $(v_1, v_4, v_2, v_5, v_6, v_3)$

Step 6:-



Set $(v_1, v_4, v_2, v_5, v_6, v_3, v_7)$

~~Answer~~

The resultant span tree are prims algorithm is

$$(v_2 \rightarrow v_5) + (v_2 \rightarrow v_4) + (v_1 \rightarrow v_4) + (v_4 \rightarrow v_6) + (v_3 \rightarrow v_6) + (v_6 \rightarrow v_7) \\ = 2 + 1 + 2 + 4 + 1 + 6 = 16 \checkmark$$

We can not consider the edges $v_4 \rightarrow v_7$, $v_4 \rightarrow v_3$, $v_5 \rightarrow v_7$, $v_1 \rightarrow v_2$ because the circuit is form so we can not consider the edges.

Applications of Minimum Spanning tree:-

network designing

telephone communication, electrical, TV cable, computer etc.